Borla

Technical Debt Plan

Student: **George Boadu Junior**

Student ID: **22427354**

Course: CSCD 602 — Advanced Software Engineering, University of Ghana

Document: Technical_Debt_Plan.pdf · Version 1.0 · 13 August 2026

1. How debt was tracked during the build

Every deliberate shortcut was written down **as the decision was made**, not reconstructed afterwards — most of the items below are quoted almost verbatim from code comments left at the exact line where the trade-off was taken (e.g. `server/src/jobs/index.ts`, `server/src/modules/auth/routes.ts`). This is the practice the course asks for: technical debt as a first-class, visible decision, not a silent shortcut discovered later.

Each item below follows **Debt → Cause → Impact → Priority → Proposed Resolution**, and is labelled as one of:

- 🟢 **Acceptable temporarily** — correct trade-off for a graded demo at pilot scale; revisit only if the project continues past this exam.
- 🟡 **Scheduled for future resolution** — fine for now, but must be fixed before real users or a meaningful scale-up.
- 🔴 **Critical** — must not reach real users in its current form.

2. Debt register

ID	Debt	Cause	Impact	Priority	Proposed resolution
TD-01	Review/reply moderation runs with **no AI key by default** — everything lands in a manual admin queue instead of being auto-screened.	No Google AI Studio account was provisioned for this build; the moderation pipeline (`server/src/ai/moderation.ts`) checks `GEMINI_API_KEY` and no-ops if absent.	Every review needs a human click before publishing; fine at demo volume (a handful of reviews), would not scale to real usage — but is explicitly the *safer* failure direction (fail-closed, not fail-open).	🟡 Scheduled	Provision a Gemini API key on Render (`GEMINI_API_KEY` env var, sync:false in `render.yaml` so it must be set manually) — the code path is already written and tested; this is a config change, not a code change.
TD-02	~~No real SMS gateway~~ — **mostly resolved**: `server/src/utils/sms.ts` sends the OTP via GiantSMS when `GIANTSMS_API_TOKEN` / `GIANTSMS_SENDER_ID` are configured, falling back to the in-app `devOtp` display only if the send fails or isn't configured. **Confirmed live in production**: `POST /api/auth/otp/request` against `https://borla.onrender.com` returned `{"delivered":true}` — GiantSMS accepted the reconstructed request shape (auth header, `from` / `to` / `msg` body) for real, not just in theory.	The HTTP contract was reconstructed from published third-party client libraries, not a first-party OpenAPI spec. Production accepted the request; actual handset delivery to a real number (the seeded demo number is a placeholder, not a live handset) has still not been visually confirmed.	Low residual risk: the gateway accepting the request end-to-end (not just returning a generic 200) is strong evidence the integration is correct, but "accepted by the gateway" and "arrived on a phone" are not identical.	🟡 Scheduled (down from 🔴 Critical — a real send now provably works at the HTTP layer)	Confirm one delivery to a real handset, then remove the `devOtp` fallback field entirely so a failed send surfaces as a retryable error instead of a security bypass.
TD-03	**Native background geolocation was not built.** Collectors only report position while their browser tab is open and in the foreground (`useGeolocation` `watchPosition`, no Capacitor/native service).	Requires an Android toolchain, a background-location plugin (commercial or complex to configure), and a physical/emulated device to validate "screen-off roaming" — none available, and a native APK isn't gradable as a "Live Application URL" per the exam's own submission format.	This is the single biggest gap from the original design's own risk register (`borla-technical-design.md` §14, risk #1) — a collector who backgrounds the app or locks their phone stops updating position, which is exactly the failure mode that erodes trust in the map.	🔴 Critical (for any real pilot) / 🟢 Acceptable for this exam	Wrap the existing shared React screens in Capacitor, add `@capacitor-community/background-geolocation`, and test on real low-end Android hardware (Tecno/Infinix/itel) per the original risk mitigation — no server-side change needed, since the API already accepts position updates from any authenticated client.
TD-04	~~No offline PWA shell~~ — **mostly resolved**: `vite-plugin-pwa` now ships a real installable PWA — manifest + icons (`client/public/icon-*.png`), a Workbox service worker precaching the app shell (12 entries, ~490KB), an explicit "Update available" banner (`registerType:'prompt'`, `src/pwa/UpdatePrompt.tsx` — never silently swaps content under an open tab) and a real "Install app" button (`src/pwa/InstallButton.tsx`, `beforeinstallprompt`). Verified: the service worker registers and activates (checked via a headless-browser `navigator.serviceWorker.getRegistrations()` call against the built `dist/`), and API/socket traffic is deliberately excluded from precaching so it's never served stale. **Still open**: households' geolocation is on-demand/foreground-only (unchanged), and there's no queued-writes-while-offline behaviour yet (e.g. "clear pin" typed while offline doesn't queue and flush on reconnect — the shell loads offline, but mutating actions still need a live connection).	The remaining gap is genuinely more work (background sync / an offline action queue), not a missing dependency.	Households still see a blank state for anything requiring a live fetch while offline (map data, requests, broadcasts) — the shell now loads, the *data* doesn't.	🟢 Acceptable (down from 🟡 Scheduled — the install/update half is done; only the offline-writes half remains)	Add a Workbox Background Sync queue (or a hand-rolled IndexedDB outbox) for "clear pin" and similarly small mutating actions so they survive a dropped connection.
TD-05	**No Redis/BullMQ.** Presence TTL, pin expiry, request timeout, and double-blind review release are all `node-cron` sweeps against Postgres directly; rate-limiting is a coarse in-memory per-IP limiter (`express-rate-limit`), not the design's per-collector notification cap.	Provisioning a managed Redis instance is an extra account/service with no functional benefit at this traffic volume; BullMQ adds retry/backoff	All scheduled-job state is **lost on process restart** (a mid-flight sweep just re-runs on the next tick — acceptable, since every sweep is idempotent by construction) and **cannot scale**	🟡 Scheduled	Add Upstash/Render Redis, move the four cron sweeps to BullMQ repeatable/delayed jobs, wire the Socket.IO Redis adapter, and replace the per-IP limiter with the design's per-collector

ID	Debt	Cause	Impact	Priority	Proposed resolution
		semantics that matter only once notification volume is real.	**past one Node instance** (Socket.IO has no Redis adapter either, so multi-instance deployments would silently drop events to users connected to a different instance).		`ratelimit:notif:{id}` counter — all four sweeps already have clean, small, idempotent bodies (`server/src/jobs/index.ts`) so the migration is mostly infrastructure, not logic rewrite.
TD-06	**Admin auth has no 2FA** — phone + password only (`bcrypt` -hashed), same JWT mechanism as household/collector, instead of the original design's "stronger auth than the field apps."	Full 2FA (TOTP/SMS) needs either the same missing SMS gateway (TD-02) or a new TOTP flow — both extra scope for an account class that, in this build, is a single seeded demo user.	An admin account compromise is higher blast radius than a household/collector one (can suspend users, remove content) — acceptable only because there is exactly one demo admin account behind a strong generated password, never exposed publicly except in the graded credentials file.	🔴 Critical (before real ops staff use it)	Add TOTP (e.g. `otplib`) gated behind the existing `role='admin'` check; store a `totp_secret` column already anticipated by the schema's extensibility (JSONB `app_config` pattern could hold it, or a dedicated column).
TD-07	The standalone `admins` table from the original design was **folded into `users.role = 'admin'`** with a nullable `password_hash` column that only admins use.	Reduces schema surface for a single-admin demo; avoids a parallel user model.	Slight modelling smell — `password_hash` is `NULL` for 99% of rows (households/collectors) — and there is no dedicated `admins.role` tier (`ops` / `moderator` / `superadmin` from the original design).	🟢 Acceptable	If multiple admin tiers are ever needed, split back into a dedicated `admins` table with its own role enum, migrated via a straightforward `INSERT ... SELECT` from `users WHERE role='admin'` .
TD-08	**Leaflet + raw OpenStreetMap raster tiles** instead of the original design's MapLibre GL + vector tiles.	Zero-config: no tile-provider account/key needed, which matters when nobody is provisioning new accounts mid-exam.	Slightly less crisp rendering and no offline vector caching; functionally equivalent for markers/popups/radius visualisation, which is all this build needs.	🟢 Acceptable	Swap the `MapView` component's tile layer for a MapLibre + vector source once a tile provider (e.g. MapTiler free tier) is chosen — the component's public interface (`center` , `points`) would not need to change.
TD-09	**No i18n beyond English**, despite the original design and the low-literacy design brief calling for Twi/Ga from day one.	Time-boxed; English-only was the fastest path to a fully functional, testable app across every screen.	Directly works against the stated accessibility goal for the target users.	🟡 Scheduled	The UI already isolates copy inside JSX rather than scattering it — introduce `react-i18next` (or the originally-planned `packages/shared/i18n`) and extract strings; start with Twi and Ga per the pilot-neighbourhood plan.
TD-10	**Automated test coverage is broad, not deep.** 31 server tests cover every route's happy path and its most important failure mode (idempotency guards, role gates, moderation gating); there is no fuzzing, no load testing, and client tests cover 2 of ~9 components.	A time-boxed exam favoured "every module has real integration tests" over "one module is exhaustively tested" — a deliberate breadth-over-depth call, consistent with §7.7 of the SRS.	Edge cases in less-tested paths (e.g. concurrent double-accept races under real network latency, malformed geo payloads) are not proven safe, only reasoned about.	🟡 Scheduled	Add property-based tests for the geospatial radius math, a concurrency test that fires accept+reject simultaneously against the same request to prove the conditional-update guard under real contention (not just sequential calls), and React Testing Library coverage for `HouseholdHome` / `CollectorHome` .
TD-11	**Photo → waste-type AI classification was not built** (§18 Job B in the original design).	Explicitly deprioritised behind moderation (Job A) per the original design's own value ranking — moderation is the safety-critical one; photo classification is a UX nicety.	Households still pick a waste-type tile manually instead of snapping a photo — a real but non-blocking gap for low-literacy accessibility.	🟡 Scheduled	Reuse the existing `aiClient` -shaped moderation module's pattern: a second thin wrapper calling Gemini's multimodal endpoint, wired to a new `POST /broadcasts` image field, client-side downscaled before upload.
TD-12	**Provider call-masking was not built** — accepted requests reveal each party's real phone number ("reveal-on-accept", the original design's own recommended v1 approach, §10 option 1).	This *is* the design's own recommended v1 trade-off, not a shortcut taken under exam pressure — kept as-is deliberately.	Real phone numbers are exchanged between two people who chose each other via an accepted request — acceptable given the low stakes described in the original design.	🟢 Acceptable	Add provider number-masking (e.g. via Hubtel) only if abuse reports or scale warrant the added per-call cost, exactly as the original design already prescribes.
TD-13	**Single Render Web Service, no staging environment, no CI pipeline beyond local `npm test` .**	Free-tier, single-service deploy was the fastest path to a real "Live Application URL" within the exam window.	A bad commit reaches production directly; no automated gate before deploy.	🟡 Scheduled	Add a GitHub Actions workflow running `npm test` on every push (config-only change, no app-code change needed) and a second Render environment for staging before promoting to the graded URL.

3. Debt NOT taken — deliberate design choices worth naming

To be explicit about the difference between *debt* (a shortcut that should eventually be paid down) and a *considered trade-off that stays*: the no-collector-binding broadcast plane, the absence of in-app payments, and reveal-on-accept-instead-of-provider-masking are not technical debt — they are the original design's own architecture, kept unchanged because they are still the right call at this scale (see `borla-technical-design.md` §1 and §10, and TD-12 above).

4. Repayment plan by phase

This maps directly onto `Project_Documentation.md` §15 (Maintenance & Future Evolution):

Phase	Items paid down
v1.1 (immediate post-submission, if continued)	TD-06 (admin 2FA) — the one remaining 🔴 Critical item, must land before any real admin touches the system; confirm one real GiantSMS handset delivery to fully close TD-02
v1.2	TD-01 (provision Gemini key in production), TD-13 (CI pipeline)
v1.3	TD-03 (native Capacitor collector app + background geolocation) — the single biggest lift, matched to the original design's own phased roadmap
v2.0	TD-05 (Redis/BullMQ + Socket.IO Redis adapter, needed once traffic outgrows one instance), TD-09 (i18n), TD-04 (offline write queue — the install/update half already shipped), TD-11 (photo classification)
Ongoing	TD-10 (expand test depth every time a new bug class is found in production)